# Project's Low-Level Design

# on

## Deploy Hospitality App to Kubernetes

Course Name: DevOps

***Institution Name***: Medicaps University – Datagami Skill Based Course

*Student Names & Enrolment Numbers:*

| Sr No | Student Name | Enrolment |
|---|---|---|
| 1 | Aeshna Preyasi | EN22CS301075 |
| 2 | Aastha Gupta | EN22CS301019 |
| 3 | Akash Ghosh | EN22CS301082 |
| 4 | Aman Gour | EN22CS301104 |
| 5 | Irifa Khaishagi | EN22CS301435 |
| 6 | Adarsh Dibey | EN22CS301037 |

*Group Name*: Group 08D1

*Project Number*: DO-08

*Industry Mentor Name*: Aashruti Shah (Program Director, Datagami)

*University Mentor Name*: Dr. Ritesh Joshi

*Academic Year*: 2025-26

# Table of Contents


1. Introduction
    a. Scope of the document
    b. Intended Audience
    c. System overview
2. System Design
    a. Application Design
    b. Process Flow
    c. Information Flow
    d. Components Design
    e. Key Design Considerations
    f. API Catalogue
3. References

# 1 Introduction

## 1.1 Scope of the document

This document describes the low-level design of the Dockerized Multi-Agent Hospitality System. It details the containerization strategy, the CrewAI orchestration pipeline, internal architecture, and the deployment workflow on AWS EC2.

## 1.2 Intended Audience

Developers, DevOps Engineers, Testers, and Project Evaluators/Faculty.

## 1.3 System Overview

The system is an Agentic AI web application that takes user travel preferences (destination, duration, budget) to generate structured itineraries. It breaks tasks into stages managed by CrewAI and powered by the Groq LLM API. The application is fully containerized using Docker to ensure consistent environments across local development and AWS production servers.

# 2 System Design

## 2.1 Application Design

Unlike a microservices approach with a database, this system transitions into a stateless, single-container architecture using Docker Layers & Containers:

- **Presentation Layer (Frontend):** Streamlit UI for capturing input and rendering the itinerary.
- **Business Logic Layer:** CrewAI Orchestrator that manages the execution flow.
- **Agent Layer:** Five specific agents (Research, Planner, Budget, Writer, Reviewer).
- **AI/Utility Layer:** Integration with Groq LLM API and DuckDuckGo Search API.

**Docker Compose Orchestration:** A single docker-compose.yml file is used to orchestrate the lifecycle, networking, and environment variables of all three containers, simulating a production-ready cluster locally.

### 2.2 Process Flow (Dockerized Execution)

- Initialization: The DevOps engineer executes the docker run command.
- Container Boot: The container starts, installs Python dependencies, and launches the Streamlit server on port 8501.
- Execution Flow: The user enters details on the UI. The Orchestrator initiates the pipeline, passing data sequentially from the Research Agent down to the Reviewer Agent.

### 2.3 Information Flow

- **Client to Container:** User interacts with the Streamlit UI via the exposed Docker port.
- **Internal Handoff:** Sequential data handoff happens entirely in-memory via the CrewAI Orchestrator.
- **Container to External APIs:** The container reaches out to the public internet via standard HTTP requests to communicate securely with the Groq API and DuckDuckGo API.

### 2.4 Component Design (Docker Architecture)

1. **The Backend Dockerfile (Multi-Stage Build)**
    - *Base Image:* python:3.10-slim (Minimizes attack surface and image size).
    - *Dependencies: Installs CrewAI, Streamlit, and integration tools via requirements.txt.*
    - *State Management: No database container is required; the system is stateless by design. The only persistent data is the app_logs.txt file, which tracks system events.*

### 2.5 Key Design Considerations (DevOps & AWS Deployment)

1. **Dynamic Secrets Management (Solving API Exhaustion):** The Groq API key is *never* baked into the Docker image. It is injected at runtime via an external .env file.

If the key exhausts, it can be updated on the EC2 instance and the container restarted without needing a rebuild.

2. **Portability & AWS Deployment:** The application is encapsulated in a Docker image, meaning deployment to an AWS EC2 instance only requires installing Docker and running a single command.
3. **Future-Proofing:** Externalizing configuration (.env) aligns perfectly with 12-Factor App methodology, making future migrations to Kubernetes (K8s) seamless.

### 2.6 API Catalogue

Because there is no internal REST backend, the system relies on external APIs.

| External Service | Authentication | Response / Purpose |
|---|---|---|
| Groq LLM API | Requires API Key (.env) | Primary intelligence provider; powers text generation for all 5 agents. |
| DuckDuckGo Search API | No strict authentication | Fetches real-world travel data for the Research Agent. |
| CrewAI Internal API | Internal Framework | Manages sequential data handoffs between agents. |

## 3 References:

1. Docker Official Documentation (Containerization, Networking).
2. Official CrewAI documentation (agent orchestration, task definitions).
3. Official Streamlit documentation (building interactive web applications).